

batata



RV32IM PG-3

Laboratório de sistemas computacionais:
Arquitetura e Organização de computadores

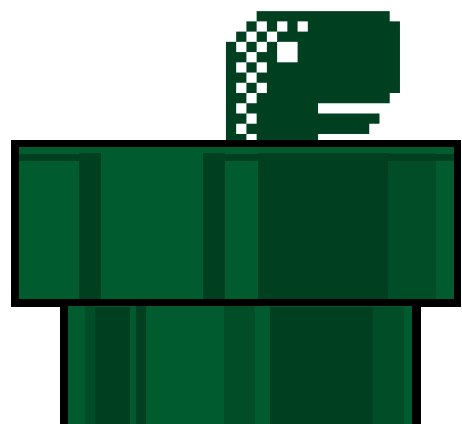
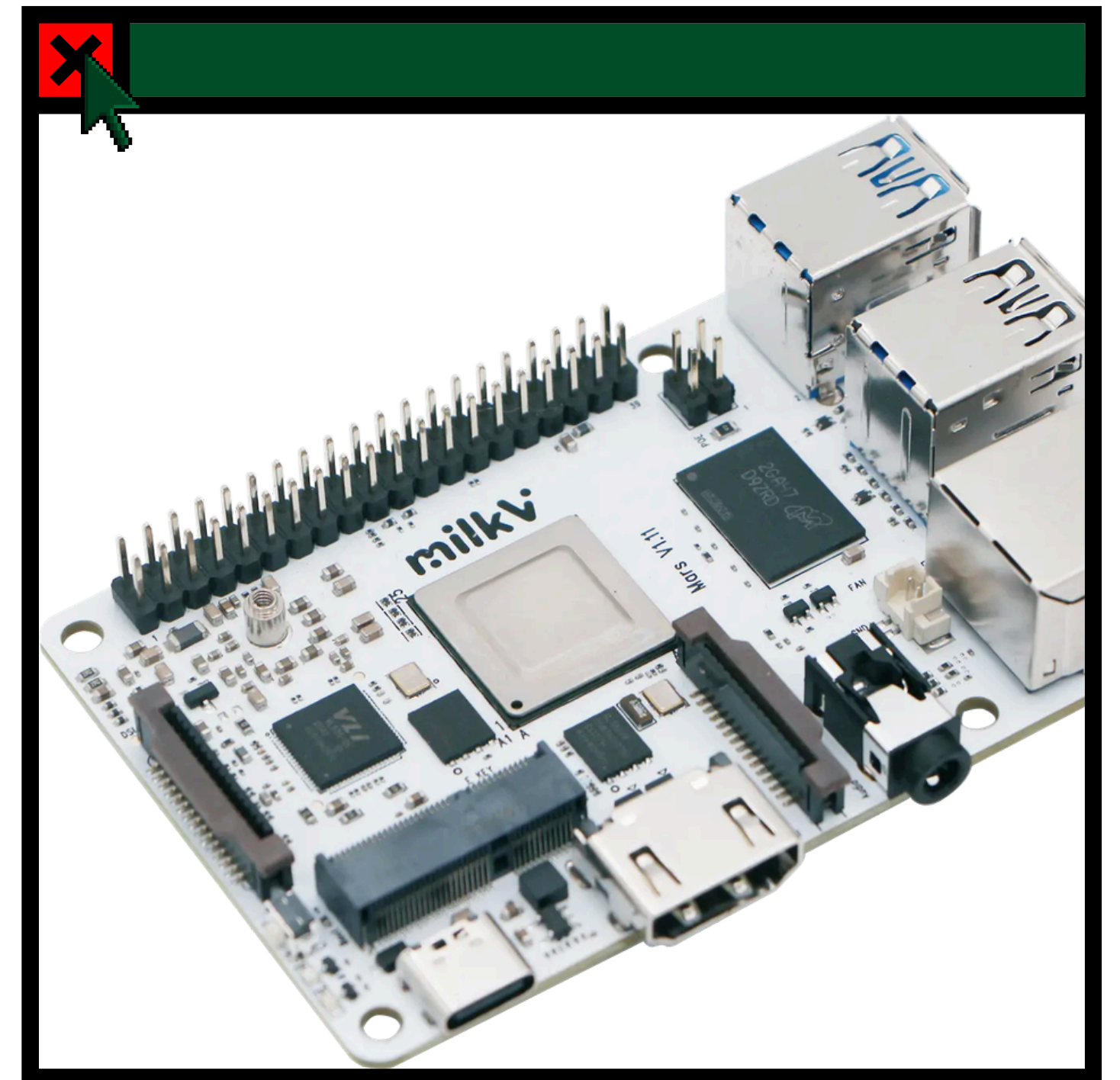
Leonardo Tomasela Leal
170291



pizza

SUMÁRIO

- Recapitulação
- Implementação control-Unit
- Código de integração
- Fibonacci - Código Testado
- Execução

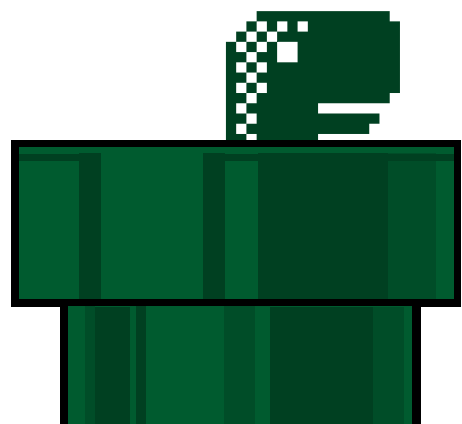


kiwi

RECAPITULAÇÃO

 Modulos implementados até o PC2

- Unidade de lógica e aritmética
- Banco de registradores
- Memória de dados
- Memória de instruções
- Unidade de desvio
- Gerador de imediatos

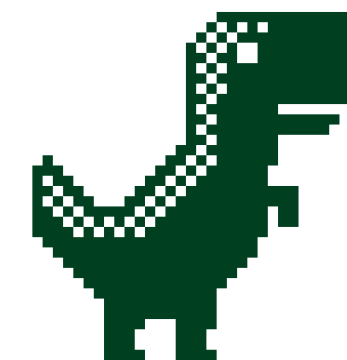
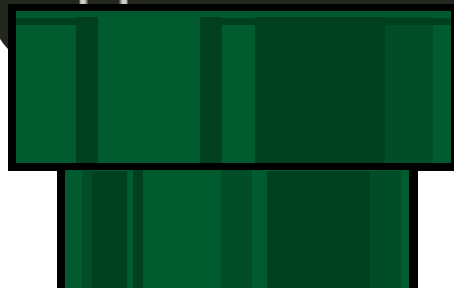


aborgue

DATA MEMORY ATUALIZADO

data_memory.sv

```
1 [...]
2
3  always_ff @(negedge clk_c) begin
4
5      if (memwrite_c) begin : mem_write
6          case (be)
7              4'b0001: data_mem[addr_i[31:2]][7:0] <=
wdata_i[7:0];
8              4'b0010: data_mem[addr_i[31:2]][15:8] <=
wdata_i[7:0];
9
10 [...]
11
```



CONTROL UNIT

```
control_unit.sv

1 import data_structures::*;
2
3 module control_unit (
4     input instruction_t inst_i,
5
6     output inst_type_t inst_type_c,
7
8     output logic memwrite_c,
9     memread_c,
10    alusel1_c,
11    alusel2_c,
12    we_c,
13    memtoreg_c,
14    lui_c,
15    branch_c,
16    jump_c,
17    pcsource_c
18 );
19
20
21 opcode_t opcode;
22 assign opcode = opcode_t'(inst_i[6:0]);
23
```

```
control_unit.sv

24 always_comb begin : control_unit
25     memwrite_c = 0;
26     memread_c = 0;
27     alusel1_c = 0;
28     alusel2_c = 0;
29     we_c = 0;
30     memtoreg_c = 0;
31     lui_c = 0;
32     branch_c = 0;
33     jump_c = 0;
34     pcsource_c = 0;
35
36     inst_type_c = OP;
```



CONTROL UNIT

```
control_unit.sv

37  case (opcode)
38      opcode_OP: begin
39          we_c = 1;
40      end
41      opcode_OP_IMM: begin
42          alusel2_c = 1;
43          we_c = 1;
44
45          inst_type_c = OPI;
46      end
47      opcode_LOAD: begin
48          memread_c = 1;
49          alusel2_c = 1;
50          we_c = 1;
51          memtoreg_c = 1;
52
53          inst_type_c = LOAD;
54      end
```

```
control_unit.sv

55      opcode_STORE: begin
56          memwrite_c = 1;
57          alusel2_c = 1;
58
59          inst_type_c = STORE;
60      end
61      opcode_BRANCH: begin
62          branch_c = 1;
63
64          inst_type_c = BRANCH;
65      end
66      opcode_JAL: begin
67          alusel1_c = 1;
68          we_c = 1;
69          jump_c = 1;
70
71          inst_type_c = JUMP;
72      end
```

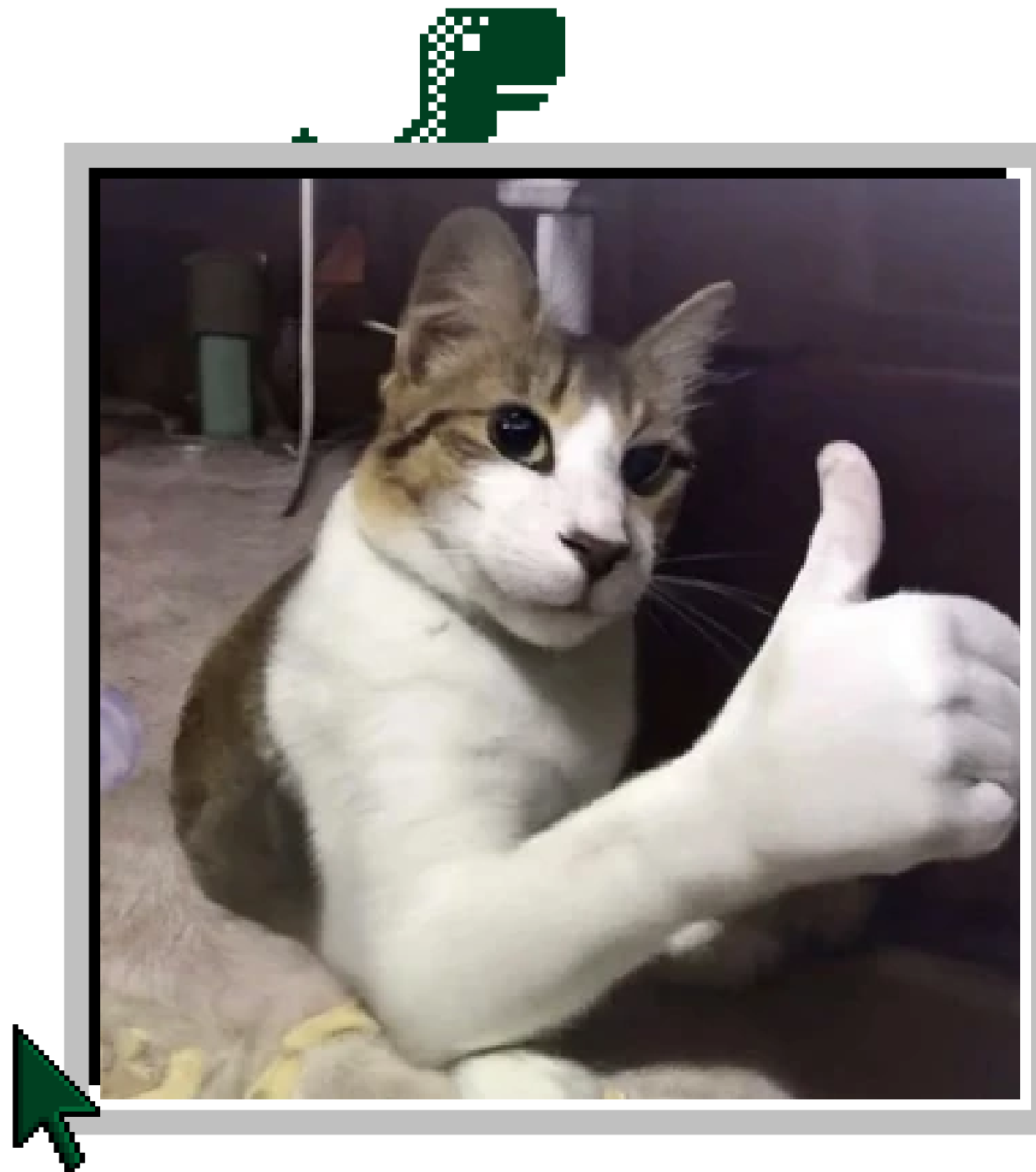


mussarela

CONTROL UNIT

```
Made with carbon-now.nvim

73  opcode_JALR: begin
74      alusel1_c = 1;
75      we_c = 1;
76      jump_c = 1;
77      pccsource_c = 1;
78
79      inst_type_c = JUMP;
80  end
81  opcode_LUI: begin
82      we_c = 1;
83      lui_c = 1;
84
85  end
86  opcode_AUIPC: begin
87      alusel1_c = 1;
88      alusel2_c = 1;
89      we_c = 1;
90
91      inst_type_c = AUIPC;
92  end
93  default: begin
94  end
95  endcase
96
97  end
98  endmodule
```

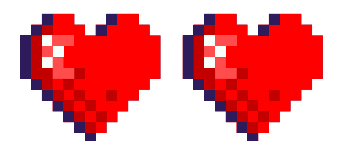
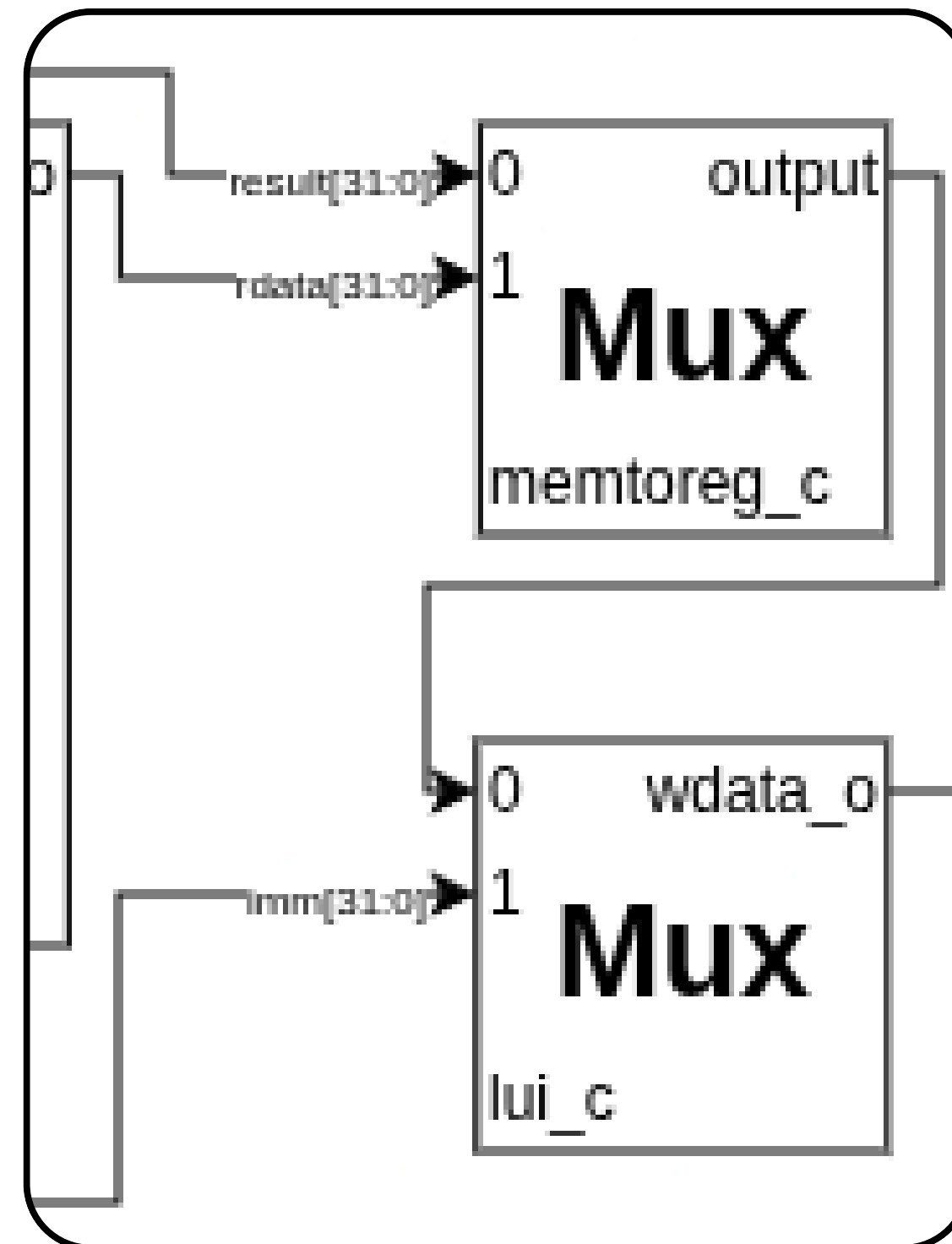


INTEGRAÇÃO - MUX

```

84  // Mux para escrever dados no
    regfile
85  always_comb begin : mux_wdata
86      wdata = 0;
87      casez ({
88          memtooreg_c, lui_c
89      })
90          2'b00: wdata = aluout;
91          2'b10: wdata = rdata;
92          2'b?1: wdata = imm;
93          default: begin
94              end
95      endcase

```

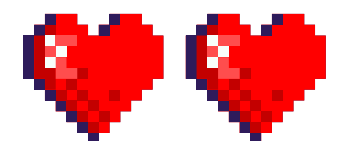
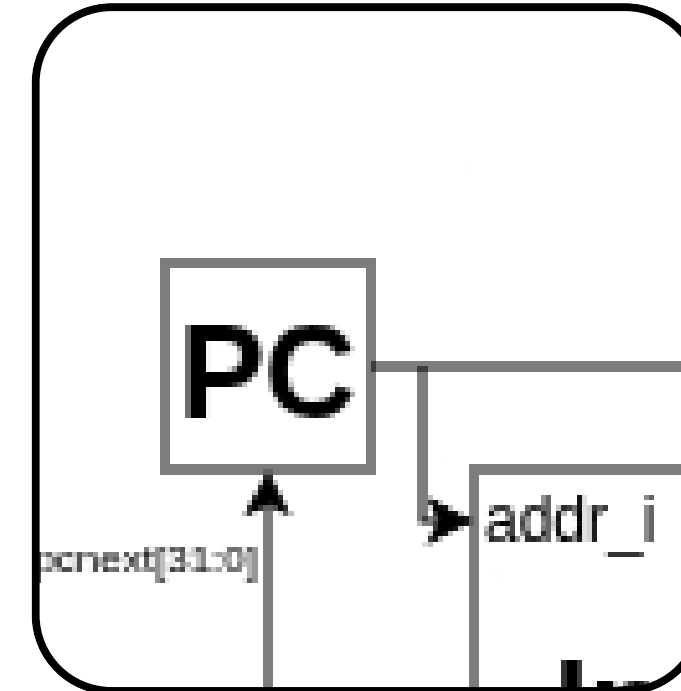


mclanche

INTEGRAÇÃO - PC

integ.sv

```
109
110  always_ff @(posedge clk_c) begin
      : pc_update
111      pc <= (RESET) ? pc_next : 0;
112  end
113
```

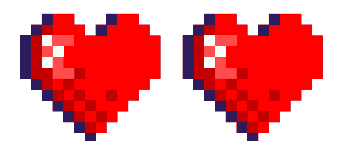


whiskas sache

INTEGRAÇÃO

```
integ.sv
1 import data_structures::*;
2
3 module integ (
4     input logic clk_c,
5     RESET
6 );
7
8 register_t rs1, rs2, pc, pc_next;
9 data_t wdata, rdata, imm, aluout;
10 logic [4:0] rdaddr, rs1addr, rs2addr;
11 funct3_t funct3;
12 funct7_t funct7;
13 instruction_t inst;
14
15 logic
16     memwrite_c,
17     memread_c,
18     alusel1_c,
19     alusel2_c,
20     we_c,
21     memtoreg_c,
22     lui_c,
23     branch_c,
24     jump_c,
25     pcsource_c;
26
27 inst_type_t inst_type_c;
28
```

```
integ.sv
29
30
31 assign rdaddr = inst[11:7];
32 assign rs2addr = inst[24:20];
33 assign rs1addr = inst[19:15];
34 assign funct3 = inst[14:12];
35 assign funct7 = inst[31:25];
36
37
```



milkshake de morango

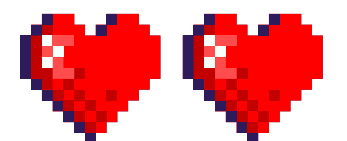
INTEGRAÇÃO

```
integ.sv
38 inst_memory inst_mem (
39     .RESET (RESET),
40     .addr_i(pc),
41     .inst_o(inst)
42 );
43
44 regfile regfile (
45     .we_c(we_c),
46     .clk_c(clk_c),
47     .wdata_i(wdata),
48     .rdaddr_i(rdaddr),
49     .rs1addr_i(rs1addr),
50     .rs2addr_i(rs2addr),
51     .rs1_o(rs1),
52     .rs2_o(rs2)
53 );
54
55 alu alu (
56     .rs1_i(rs1),
57     .rs2_i(rs2),
58     .imm_i(imm),
59     .pc_i(pc),
60     .funct3_i(funct3),
61     .funct7_i(funct7),
62     .inst_type_c(inst_type_c),
63     .alusel1_c(alusel1_c),
64     .alusel2_c(alusel2_c),
65     .aluout_o(aluout)
66 );
```

```
integ.sv
68 data_memory data_mem (
69     .clk_c(clk_c),
70     .memwrite_c(memwrite_c),
71     .memread_c(memread_c),
72     .funct3_i(funct3),
73     .addr_i(aluout),
74     .wdata_i(rs2),
75     .rdata_o(rdata)
76 );
77
78 immediate_generator imm_gen (
79     .inst_i(inst),
80     .inst_type_c(inst_type_c),
81     .imm_o(imm)
82 );
```

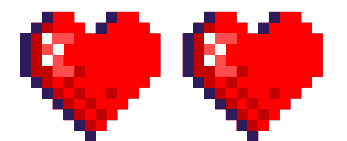
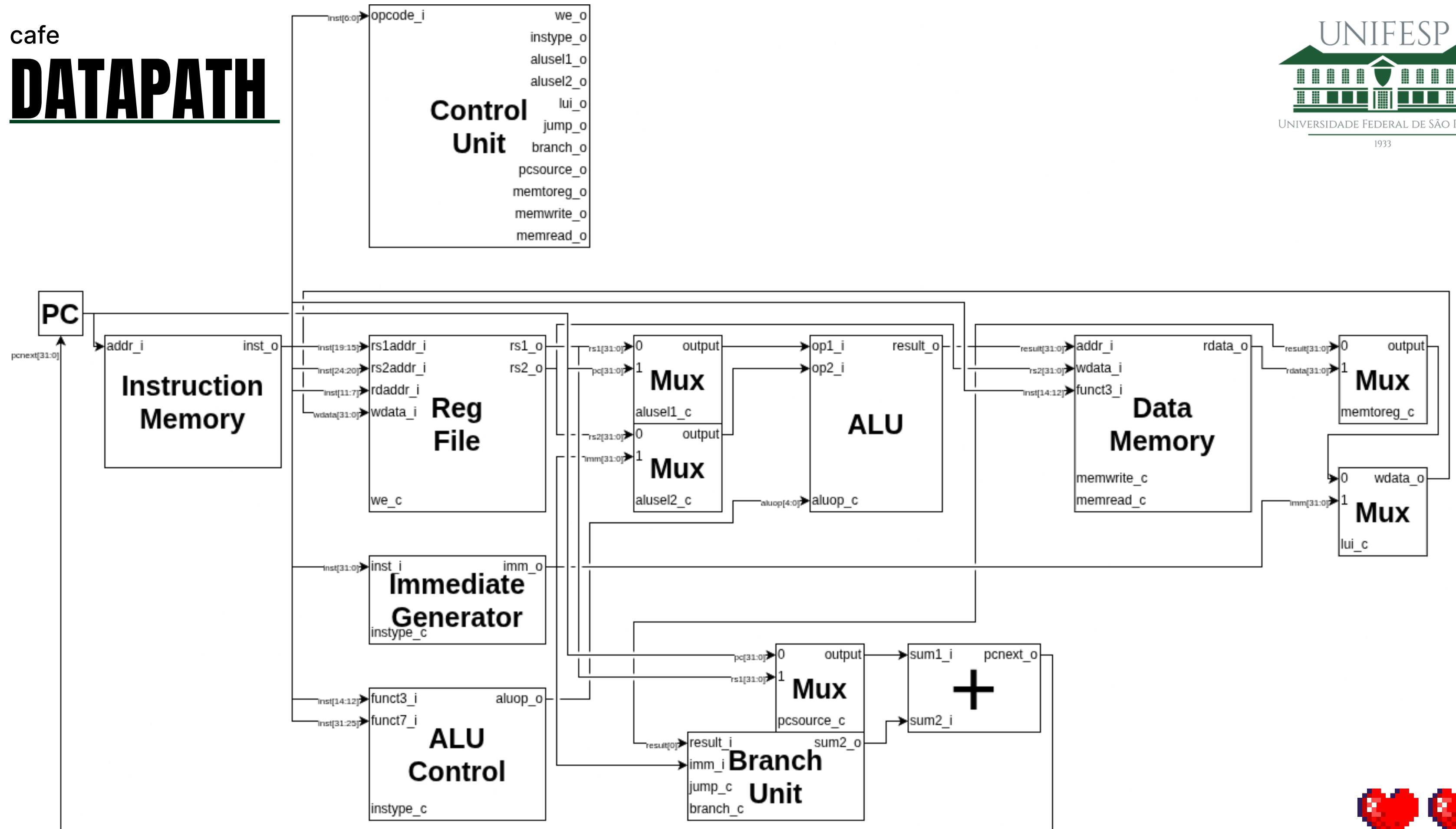
```
integ.sv
99 branch_unit bu (
100     .rs1_i(rs1),
101     .pc_i(pc),
102     .aluout_i(aluout),
103     .imm_i(imm),
104     .jump_c(jump_c),
105     .branch_c(branch_c),
106     .pc_next_o(pc_next),
107     .pcsource_c(pcsource_c)
108 );
```

```
integ.sv
144 control_unit cu (
145     .memwrite_c(memwrite_c),
146     .memread_c(memread_c),
147     .alusel1_c(alusel1_c),
148     .alusel2_c(alusel2_c),
149     .we_c(we_c),
150     .memtoreg_c(memtoreg_c),
151     .lui_c(lui_c),
152     .branch_c(branch_c),
153     .jump_c(jump_c),
154     .pcsource_c(pcsource_c),
155     .inst_i(inst),
156     .inst_type_c(inst_type_c)
157 );
```



cafe

DATAPATH



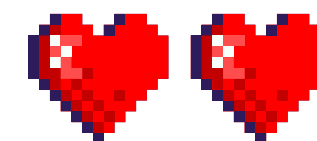
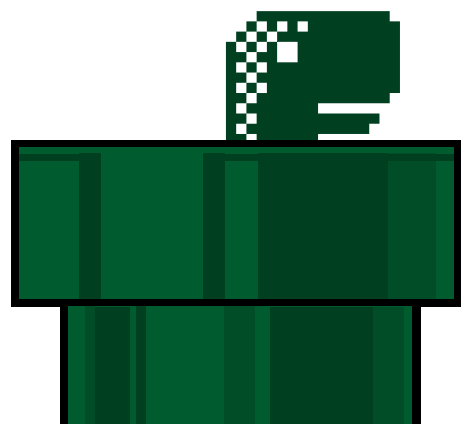
SEQUÊNCIA DE FIBONACCI - COMPILADOR

script.fish

```
1 read -l -P "Qual o arquivo de origem?" origem
2 read -l -P "Qual o arquivo de destino?" destino
3
4 riscv64-elf-gcc -march=rv32im -mabi=ilp32 -
  nostartfiles -T link.ld $origem -o output.elf
5 riscv64-elf-objcopy -j .text -O binary
  output.elf inst.bin
6 python3 -c "import sys;
  data=open('inst.bin','rb').read();
  [print(f'{int.from_bytes(data[i:i+4],
  \"little\"):08x}') for i in range(0, len(data),
  4)]" >$destino
7 rm output.elf
8 rm inst.bin
```

inst.hex

```
1 00000293
2 00100313
3 00700513
4 00100e13
5 00ae5c63
6 001e0e13
7 00030e93
8 00530333
9 000e8293
10 fedff06f
11 00030313
12 00602023
13 00002f03
```



SEQUÊNCIA DE FIBONACCI - CÓDIGO

fibasm.S

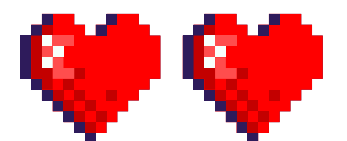
```
1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```

Valores iniciais

Elemento a ser calculado

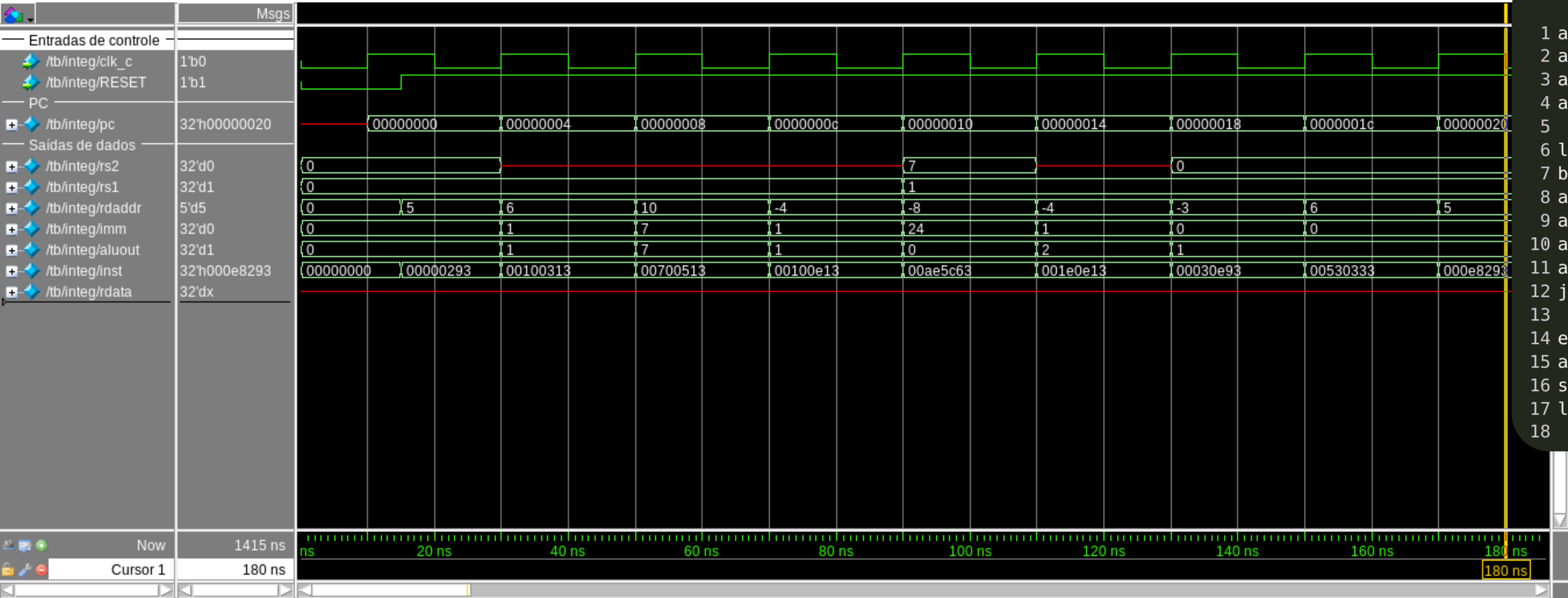
Calculo dos elementos

Guarda e carrega da memória

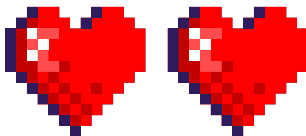


rucula

EXECUÇÃO

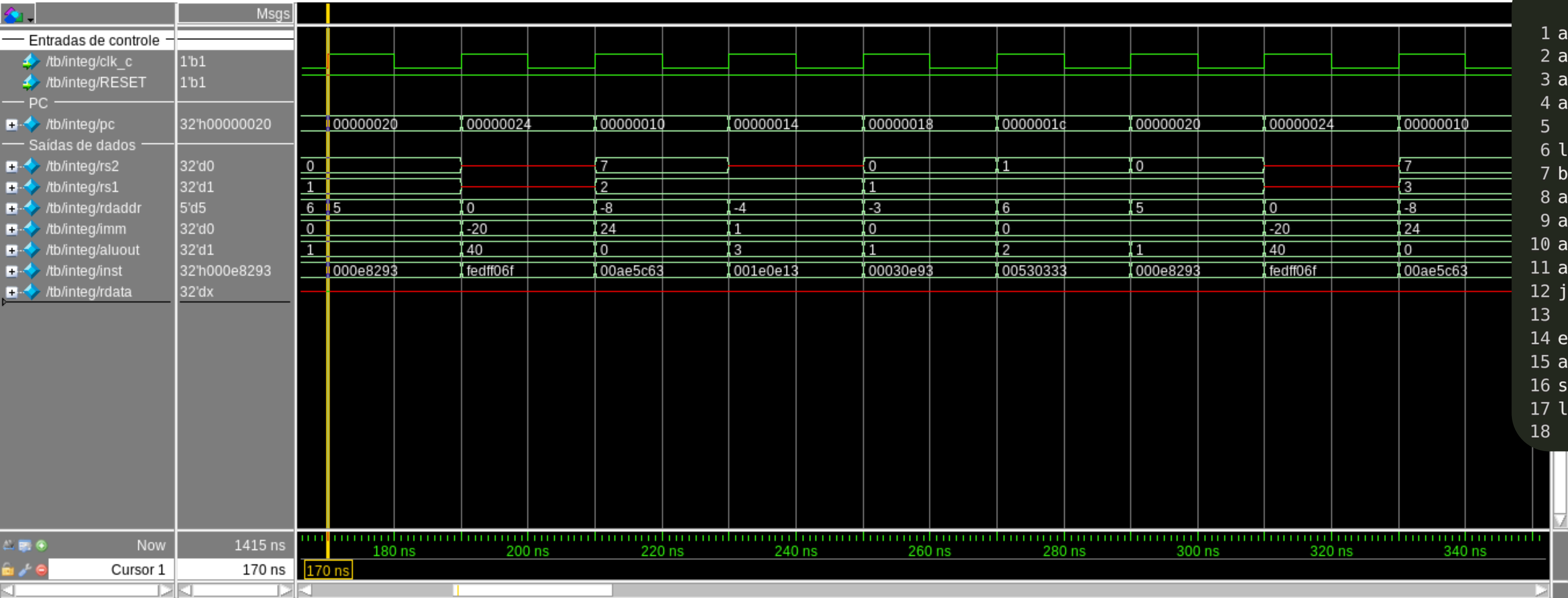


```
fibasm.S
1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```

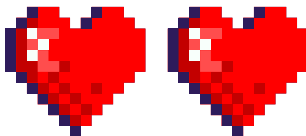


feijoada

EXECUÇÃO

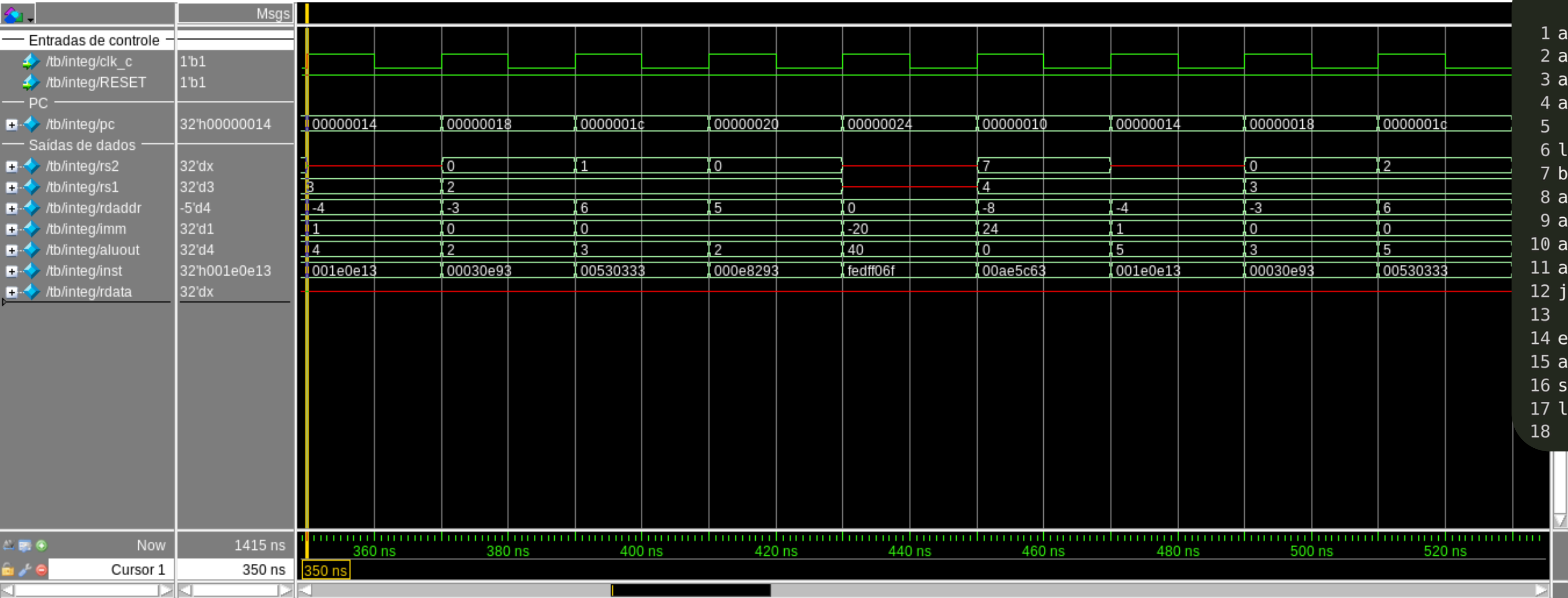


```
fibasm.S
1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```



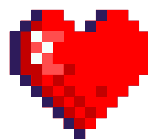
nuggets

EXECUÇÃO



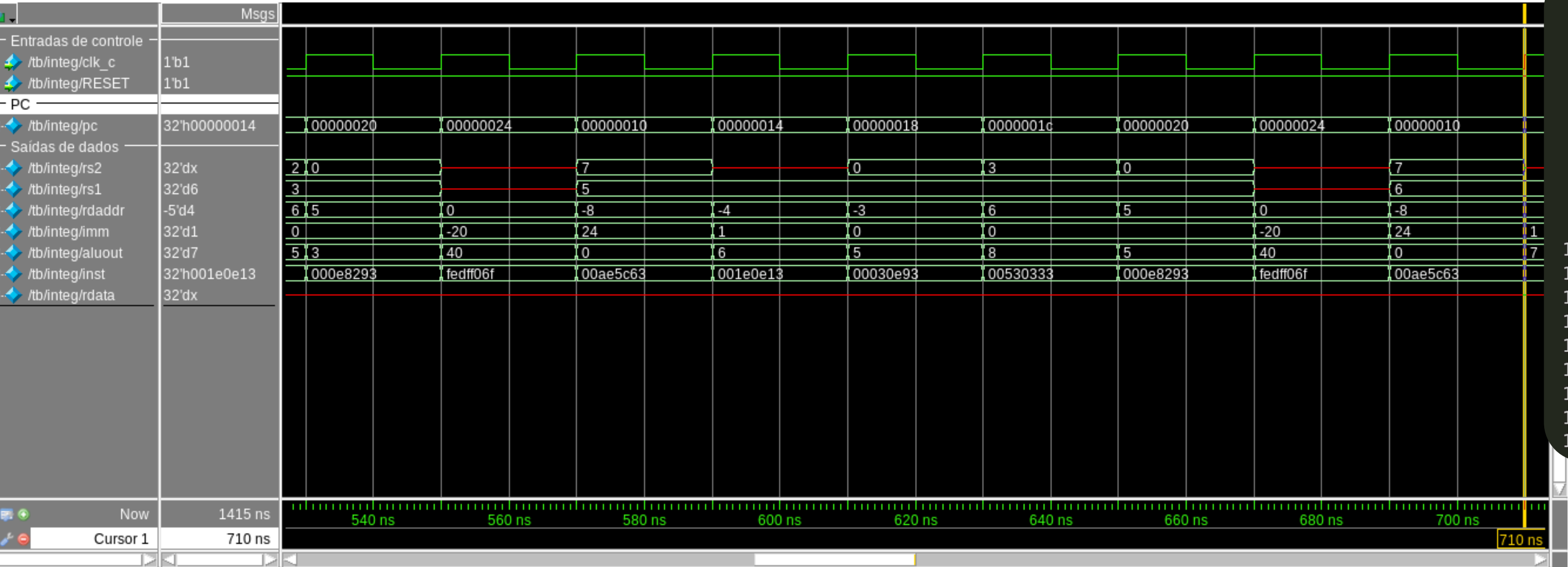
```
fibasm.S

1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```



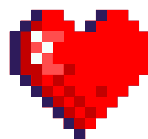
peixe

EXECUÇÃO

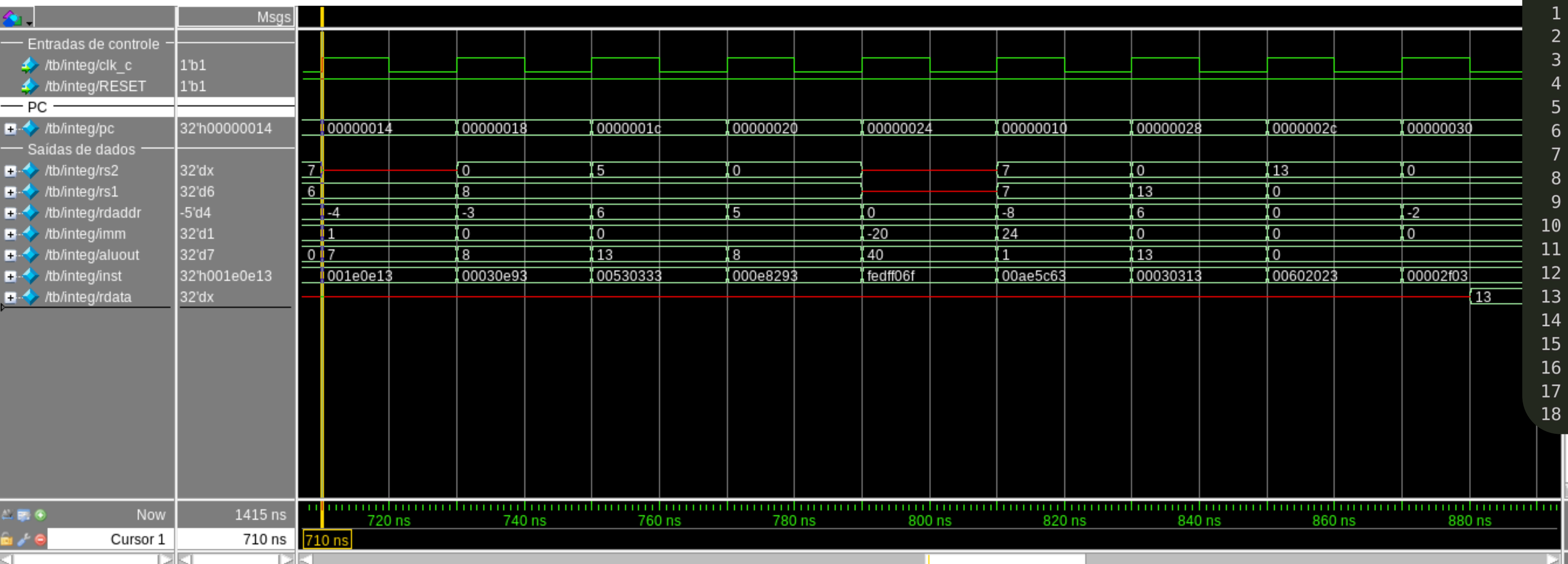


```
fibasm.S

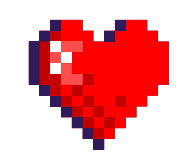
1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```



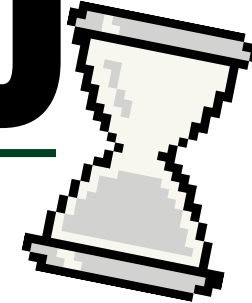
whey EXECUÇÃO



```
fibasm.S
1 addi x5, x0, 0
2 addi x6, x0, 1
3 addi x10, x0, 7
4 addi x28, x0, 1
5
6 loop:
7 bge x28, x10, end
8 addi x28, x28, 1
9 addi x29, x6, 0
10 add x6, x6, x5
11 addi x5, x29, 0
12 jal x0, loop
13
14 end:
15 addi x6, x6, 0
16 sw x6, 0(x0)
17 lw x30, 0(x0)
18
```



OBRIGADO PELA ATENÇÃO



Fontes e referências

- **Flowchart Maker & Online Diagram Software.** Disponível em: <https://app.diagrams.net/#>. Acesso em: 1 abr. 2026.
- **RISC-V Foundation.** Disponível em: <https://riscv.org/>. Acesso em: 1 abr. 2026.
- **RISC-V Reference RISC-V Instruction Set.** Disponível em: https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/notebooks/RISCV/RISCV_CARD.pdf. Acesso em: 1 abr. 2026.
- **RISC-V.** Disponível em: <https://en.wikipedia.org/wiki/RISC-V>. Acesso em: 1 abr. 2026.
- **Five EmbedDev.** Disponível em: <https://five-embeddev.com/>. Acesso em: 1 abr. 2026.
- PATTERSON, D. A.; HENNESSY, J. L. **Computer organization and design RISC-V edition : the hardware software interface.** Amsterdam: Morgan Kaufmann, 2021.
- **Carbon Now.** Disponível em: <https://carbon.now.sh/>. Acesso em: 17 jun. 2026

